

CLI Deep Dive

터미널에서 Codex를 완전히 활용하기

One agent for everywhere you code - CLI, IDE, App, Web

Codex Deep Dive Workshop (2-Day) - Module 2 / CLI Deep Dive

Choi, WooHyung

Prin. Solutions Architect

AWS Korea

whchoi@amazon.com



MODULE 02

CLI Deep Dive

터미널에서 Codex를 완전히 활용하기



이 모듈의 목표

이 모듈에서 얻어갈 네 가지

GOAL

이 모듈을 마치면 CLI로 일상 개발 작업을 자신 있게 위임하고, 세션을 정밀하게 제어하며, 자동화로 확장할 수 있습니다.

SETUP

설치와 인증

CLI 설치, 로그인, 기본 환경을 구성합니다

OPERATE

세션 운영

대화형 작업과 컨텍스트 제어를 익힙니다

CONTROL

정밀 제어

옵션, slash 명령, 모델 전환을 다룹니다

SCALE

확장

비대화형 exec로 자동화에 연결합니다

70 min

강의 55 + 랩 15

TUI

대화형 세션

exec

비대화형 실행

Slash

세션 제어 명령

CLI Overview

터미널 네이티브 코딩 에이전트

CODEX CLI

Codex CLI는 터미널에서 직접 실행되는 경량 에이전트로, 로컬 코드베이스를 대상으로 대화형 또는 비대화형으로 작업합니다.

INTERACTIVE

대화형 TUI

터미널 UI에서 대화하며 작업을 단계적으로 진행합니다

EXEC

비대화형 실행

스크립트와 CI에서 한 번에 작업을 지시합니다

LOCAL

로컬 우선

코드를 로컬에서 읽고 수정하며 셀을 직접 실행합니다

OPEN

오픈소스

Rust 네이티브, 오픈소스로 투명하게 검증 가능합니다

TUI

대화형 인터페이스

Headless

비대화형 모드

Rust

경량 네이티브

OSS

Apache 2.0

기본 사용 흐름

하루의 작업 리듬

1 cd repo

프로젝트 진입

2 codex

세션 시작

3 프롬프트

작업 지시

4 승인

diff 검토

5 commit

결과 반영

Terminal - example session

```
$ cd my-app
```

```
$ codex
```

```
> auth.test.ts의 실패하는 테스트를 고치고 원인을 설명해줘
```

```
... 테스트 실행, 원인 분석 후 패치를 제안하고 승인을 요청
```

RHYTHM 지시, 검토, 승인, 커밋의 리듬이 CLI 사용의 기본 단위입니다. 작업을 작게 쪼갤수록 이 리듬이 빨라집니다.

5 Steps

기본 작업 리듬

diff

적용 전 검토

승인

사람이 게이트

작게

검증 가능한 단위

CLI 핵심 기능 (1)

코드 작업과 상호작용

CODE WORK

코드 작업

- 자연어 프롬프트로 작업 지시
- 파일 읽기, 쓰기, apply_patch 기반 편집
- 셸 명령 실행과 결과 관찰
- 테스트와 빌드 자동 실행, 반복 수리

INTERACTION

상호작용

- 세션 컨텍스트 유지와 멀티턴 대화
- diff 기반 변경 미리보기
- 승인 모드로 안전하게 단계 진행
- 이미지 입력으로 스크린샷 기반 작업

PLAN MODE v0.94부터 기본 활성화된 Plan 모드는 실행 전에 읽을 파일, 변경, 실행 명령을 구조화된 계획으로 먼저 보여줍니다. 작은 요청에 절반을 다시 쓰는 사고를 막아줍니다.

Multi-turn

맥락 유지 대화

Patch

정밀한 파일 편집

Self-fix

테스트 기반 수리

Plan

실행 전 계획 제시

CLI 핵심 기능 (2)

고급 기능과 생산성

RESUME

세션 재개

이전 세션을 이어받아 작업 맥락을 복원합니다

MCP

도구 확장

MCP 서버로 외부 도구와 데이터 활용합니다

MODEL

모델 전환

세션 중 모델과 reasoning effort를 바꿉니다

PROFILE

프로파일

config 프로파일로 작업별 설정을 전환합니다

PRODUCTIVITY

생산성 기능 한눈에

- @파일 참조로 특정 파일을 컨텍스트에 추가, /review로 머지 전 자동 리뷰
- --add-dir로 모노레포의 다른 패키지를 읽기 참조, /undo로 변경 묶음 롤백

resume

맥락 복원

MCP

외부 도구 연결

/model

세션 중 전환

--profile

설정 프리셋

Command Line Options

주요 명령행 옵션

01	codex	대화형 세션 시작	시작
02	codex "<task>"	작업을 한 번에 지시	단발 작업
03	codex exec	비대화형 헤드리스 실행	자동화
04	codex resume	이전 세션 이어가기	연속성
05	codex -i <image>	스크린샷, 목업을 입력으로 첨부	멀티모달
06	codex cloud	클라우드 작업 제출과 적용	원격 실행
07	--model / --cd / --config	모델, 디렉토리, 설정 오버라이드	플래그

2 진입

TUI 또는 exec

resume

세션 연속성

-i

이미지 입력

--config

단발성 오버라이드

비대화형 실행

자동화를 위한 codex exec

Terminal - headless

```
# 헤드리스 실행, 스크립트와 CI에 적합. 종료 코드를 반환
$ codex exec "Bump all dependencies and run tests"

# 출력을 파일로 캡처
$ codex exec "Generate changelog" > out.md
# 작업 디렉토리 지정, --full-auto로 무인 실행
$ codex exec --cd ./service --full-auto "Add health check"
```

CI READY exec는 작업을 받아 실행하고 종료 코드와 함께 끝나는 프로세스입니다.
사람 개입이 없으므로 CI 파이프라인, 야간 배치, 대량 리포 작업에 바로 들어갑니다.

Headless

TUI 없는 실행

Exit code

파이프라인 판정

stdout

출력 캡처 가능

Lab B

Day 2에서 CI 통합

Slash Commands

세션 내 제어 명령

SESSION

세션 운영

- /help, 사용 가능한 명령 목록
- /clear, 세션 컨텍스트 초기화
- /compact, 컨텍스트 수동 압축
- /undo, 마지막 변경 묶음 되돌리기
- /debug-config, 설정 값과 출처 확인

WORK

작업 제어

- /model, 모델과 reasoning effort 변경
- /diff, 현재까지의 변경 확인
- /review, 기준 브랜치 대비 자동 리뷰
- /approvals, 승인 모드 전환
- /mcp와 /skills, 확장 도구 관리

FIRST AID 설정이 기대와 다르게 동작하면 /debug-config부터 보십시오.
어떤 값이 어느 파일에서 왔는지 보여주는 가장 빠른 진단 도구입니다.

11+

주요 slash 명령

/review

머지 전 필수 습관

/undo

묶음 단위 롤백

/compact

긴 세션 관리

파일 참조와 컨텍스트

컨텍스트 제어 기법

Terminal - session input

```
# 특정 파일을 컨텍스트에 명시적으로 추가
> Review @src/auth/login.ts for security issues

# 여러 파일 동시 참조
> Compare @config/dev.yaml and @config/prod.yaml
# 이미지(스크린샷) 첨부 후 작업
> Implement this UI from the attached screenshot
```

WHY IT MATTERS 명시적 파일 참조는 에이전트가 올바른 맥락에 집중하게 하여 정확도를 높이고 토큰 낭비를 줄입니다.
큰 리포일수록 효과가 큼니다.

@file

명시적 컨텍스트

다중

여러 파일 비교

이미지

스크린샷 작업

정확도

집중된 맥락 효과

TUI 생산성 기능

터미널 UI의 파워 기능

Terminal - power features

```
> /review    # 머지 전 변경을 에이전트가 자동 리뷰  
$ codex -i ./mockup.png "이 목업대로 컴포넌트를 구현해줘"  
$ git log --oneline -20 | codex -p "릴리스 노트 초안 작성"
```

REVIEW

/review

기준 브랜치 대비 diff를 자동
분석해 머지 전 점검합니다

IMAGE

이미지 입력

스크린샷과 목업을 프롬프트와
함께 전달합니다

PIPE

파이프 연계

셀 출력을 -p로 넘겨 컨텍스트
로 사용합니다

UNDO

/undo

멀티 파일 변경도 작업 묶음 단
위로 되돌립니다

/review

PR 전 자동 점검

-i

목업, 스크린샷

-p

stdin 파이프

/undo

안전한 실험

Steer Mode

실행 중에 방향을 조정하는 협업

STEER

에이전트가 작업을 실행하는 도중에도 메시지를 보내 방향을 조정합니다.

처음부터 다시 시작할 필요 없이 진행 중인 흐름에 새 지침이 반영되며, v0.98부터 기본 활성화되었습니다.

1 Run

에이전트 작업 진행

2 Steer

실행 중 추가 지시

3 Adjust

방향 즉시 반영

4 Continue

중단 없이 계속

ENTER 즉시 개입

실행 중 메시지를 바로 전달해 현재 작업의 방향을 수정합니다.
끝난 작업은 유지됩니다

TAB 다음 턴 예약

진행을 끊지 않고 다음 지시를 큐에 올려 둡니다.
공유 투두 리스트에 메모하는 감각입니다

Enter

현재 작업 수정

Tab

다음 작업 예약

무중단

맥락 보존 협업

구체적

지시가 정확할수록 효과

모델과 추론 강도 전환

세션 중 즉시 전환

Terminal - switch

```
# 세션 도중 모델과 강도 변경
> /model
Select model: gpt-5.3-codex (high reasoning)
# 또는 시작 시 플래그로 지정
$ codex --model gpt-5.1-codex-mini "Format and rename"
```

QUICK

빠른 작업

낮은 강도와 Mini로 단순 수정을 빠르게 처리합니다

DEEP

깊은 작업

높은 강도와 5.3으로 복잡한 리팩터링을 정확히 수행합니다

SWITCH

유연한 전환

작업 성격이 바뀌면 세션 중 즉시 전환합니다

/model

세션 중 전환

--model

시작 시 지정

low

포맷, 이름 변경

high

리팩터링, 디버깅

세션 재개와 이력

끊김 없는 작업 연속성

Terminal - resume

```
# 세션 목록에서 골라 재개
$ codex resume
# 가장 최근 세션을 바로 이어가기
$ codex resume --last
# 특정 세션 ID로 재개
$ codex resume <session-id>
```

CONTINUITY 재개된 세션은 대화 전체와 읽은 파일, 내린 결정까지 보존합니다.
노트북이 꺼져도, 어제의 디버깅을 오늘 이어가도 맥락 손실이 없습니다.

resume

세션 복원

--last

최근 세션 바로

전체 보존

대화, 파일, 결정

이력

세션 목록 관리

CLI에서 MCP 사용

MCP 서버로 기능 확장

1 Configure

config에 서버 등록

2 Connect

세션에서 연결

3 Discover

도구 자동 탐색

4 Use

에이전트가 호출

~/.codex/config.toml

```
# CLI 한 줄 등록 (config에 자동 기록)
$ codex mcp add github -- npx -y @modelcontextprotocol/server-github
[mcp_servers.github]
command = "npx"
args    = ["-y", "@modelcontextprotocol/server-github"]
```

OAUTH + DEEP DIVE OAuth가 필요한 서버는 `codex mcp login <name>`으로 인증하고 토큰은 키체인에 저장됩니다.
서버 구성과 보안은 Day 2의 Module 6과 8에서 깊이 다룹니다.

mcp add

한 줄 등록

STDIO

로컬 프로세스

HTTP

원격 서버

/mcp

세션 내 관리

환경 변수와 플래그

동작 미세 조정

01	OPENAI_API_KEY	API 키 기반 인증 (CI, SDK)	인증
02	CODEX_HOME	config와 상태 디렉토리 위치 변경	환경
03	--cd <path>	실행 작업 디렉토리 지정	범위
04	--add-dir <path>	추가 읽기 디렉토리 (모노레포)	범위
05	--config k=v	단발성 설정 오버라이드	설정
06	--full-auto	자동 승인 실행, 격리 환경 전용	주의

MONOREPO codex --cd apps/frontend --add-dir ../backend 패턴이면 프론트엔드를 수정하면서 백엔드 타입을 읽어 API 호출을 일치시킵니다.

ENV	--add-dir	쓰기 1곳	격리
키와 홈 디렉토리	읽기 전용 추가	--cd 기준 디렉토리	full-auto 사용 조건

실전 시나리오

현장에서 자주 쓰는 패턴

COMMON TASKS

자주 쓰는 작업

- 버그 수리, 실패 로그로 재현과 진단, 수정
- 테스트 작성, 커버리지 낮은 모듈 보강
- 리팩터링, 중복 추출과 구조 개선
- 문서화, 주석과 README 보강

TIPS

효과적 사용 팁

- 작업을 작고 검증 가능한 단위로 쪼개기
- 변경마다 테스트를 돌려 회귀 방지
- diff를 꼼꼼히 검토한 뒤 승인
- AGENTS.md로 컨벤션을 미리 알려주기

MINDSET 에이전트는 빠른 타이피스트와의 페어 프로그래밍입니다.
방향은 사람이 잡고, 검증은 테스트가 하고, 반복 노동은 에이전트가 맡습니다.

Bug fix

로그 기반 수리

Tests

커버리지 보강

Refactor

구조 개선

Docs

문서 자동화

CLI 트러블슈팅

자주 만나는 문제와 1차 대응

01	인증 실패	codex login 재실행, 네트워크와 프록시 확인	인증
02	변경이 적용 안 됨	승인 모드 확인, 샌드박스 쓰기 권한 점검	권한
03	느린 응답	reasoning effort 낮추기, 컨텍스트 정리	성능
04	MCP 도구 안 보임	config 서버 등록과 명령 경로 확인	확장
05	컨텍스트 초과	/compact 또는 /clear 후 핵심만 재참조	세션

DIAGNOSE FIRST 원인이 불명확하면 /debug-config로 설정 출처부터 확인하십시오.
문제의 절반은 의도하지 않은 설정 계층에서 옵니다.

5 증상

현장 빈출 이슈

login

인증 1차 조치

effort

속도 1차 조치

/compact

긴 세션 1차 조치

Module 2 요약

핵심 정리

ONE LINE

CLI는 대화형 TUI와 비대화형 exec 두 모드를 제공하며, @참조와 slash 명령, Steer Mode가 세션을 정밀하게 제어합니다.

TWO MODES

TUI / exec

대화형으로 다듬고 비대화형으로 자동화합니다

CONTROL

정밀 제어

@참조, slash 명령, /model 전환을 익혔습니다

STEER

실행 중 협업

Enter 즉시 개입, Tab 다음 턴 예약입니다

CONTINUITY

맥락 연속

resume과 MCP로 세션을 잇고 확장합니다

TUI+exec

두 가지 동작 모드

11+

slash 명령

Enter/Tab

Steer 두 키

Next

App과 IDE

Mini Lab 2 (15분)

세션을 정밀하게 제어하기

MISSION

@파일 참조, slash 명령, 모델 전환, 비대화형 exec까지 이 모듈의 제어 기법 네 가지를 연속으로 사용합니다.

Terminal - mini lab 2

```
# Step 1. 파일을 지정해 리뷰 요청
> Review @src/<핵심파일> for bugs and edge cases
# Step 2. /diff로 변경 확인, /model로 강도 변경 후 같은 질문
> /diff /model
# Step 3. 세션 밖에서 exec 한 번 실행
$ codex exec "Summarize this repo in 5 bullets"
```

CHECKPOINT @참조 리뷰, effort 변경 전후 비교, exec 출력까지 확인하면 통과입니다.
effort에 따라 응답 깊이가 어떻게 달라지는지 관찰해 보세요.

15 min

제한 시간

3 steps

참조, 전환, exec

비교

low vs high 응답

Thank you!

Choi, WooHyung / Prin. Solutions Architect / AWS Korea

whchoi@amazon.com

<https://linkedin.com/in/woohyungchoi>

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved. Amazon Confidential and Trademark.

